

Лабораторна робота №4. MCP-сервер, мультиагентна конфігурація та оцінювання

25 серпня 2026 р.

Мета

Опублікувати власний MCP-сервер лише для читання, порівняти одиночного агента з мультиагентною конфігурацією на одному наборі задач і показати числами, чи виправдана друга архітектура.

Робота спирається на лекції 13, 16, 17. Аудиторний час — 4 години, самостійна робота — 8 годин.

Теоретичні відомості

Доки кожен застосунок пише власний конектор до кожної системи, кількість інтеграцій росте як добуток $M \cdot N$. Спільний протокол розриває добуток на суму $M + N$: пишуться M клієнтів і N серверів. Виграш з'являється не одразу — на малих числах суми й добутки майже збігаються, і накладні витрати протоколу помітніші за економію.

- **Підключення сервера є розширенням межі довіри, а не інтеграцією.** Сервер отримує ваш контекст і повертає текст, який стане для моделі інструкцією. Тому сервер лише для читання пишуть першим, а права додають після того, як запрацював журнал аудиту.
- **Протокол розділяє два рівні помилок.** Невідомий інструмент і аргументи, що не пройшли схему, — це протокольні помилки JSON-RPC, корисні застосунку. Семантично хибні дані схему проходять і повертаються як результат із `isError: true`, придатний для самовиправлення моделі. Плутати ці рівні означає або ховати збій від застосунку, або позбавляти модель зворотного зв'язку.
- **Друга архітектура множить не лише якість.** Разом із другим агентом ростуть канали координації, витрата токенів і кількість режимів відмови. Справжньою причиною виграшу є ізоляція контексту:

якщо кожен агент тягне те саме, що бачать інші, витрата росте квадратично, а користі немає.

- $\text{pass}@k$ і pass^k **рахуються на тій самій множині прогонів і дають протилежні відповіді**. Перша росте з k і описує дослідника, який має право перезапустити; друга спадає і описує продакшн, де спроба одна. Середня точність не є надійністю: розрив між ними задає розподіл успіху за задачами, а не середнє.
- **Інтервал Вілсона описує кожну частку окремо і не є тестом різниці**. Висновок «інтервали перекриваються, отже конфігурації однакові» некоректний. Різницю оцінюють парно: обидві конфігурації міряються на тому самому наборі, для кожної задачі береться різниця, і 95 % інтервал будується вже для середньої різниці. Три прогони тих самих сорока задач не дають ста двадцяти незалежних спостережень: одиниця ресемплінгу — задача.
- **Суддя на основі моделі має виміряні упередження** — до позиції, до багатослівності й до власного стилю. Тому звітують не сиру частку збігів із людиною, а каппу: суддя, який завжди каже «А», має високу сиру узгодженість і нульову інформативність.

Корисні посилання для ознайомлення:

- Model Context Protocol: [з чого почати](#) та [Server: Tools](#);
- [Довідкові сервери протоколу](#) і [офіційний реєстр](#);
- Anthropic, [How We Built Our Multi-Agent Research System](#);
- Cognition, [Don't Build Multi-Agents](#) — протилежна позиція;
- OpenTelemetry, [Semantic Conventions for Generative AI](#).

Порядок виконання

Позначка *ауд.* означає, що крок виконується на занятті, *сам.* — самостійно. Складання набору задач, 240 агентних прогонів і калібрування судді свідомо винесені на домашній етап.

1. **Реалізувати MCP-сервер лише для читання** (60 хв, ауд.) з двома інструментами над локальними даними (наприклад, пошук у корпусі з лабораторної 2 і читання довідкової таблиці). Транспорт stdio, схеми аргументів і результату описані явно.

Перевірка: у stdio-режимі жоден `print()` не йде в `stdout` — це найчастіший баг, і він ламає протокол мовчки. Логи тільки в `stderr`.

2. **Реалізувати клієнта й аудит-лог** (40 хв, ауд.): під'єднатися до сервера, отримати перелік інструментів, викликати їх.

Перевірка: у логу є час, інструмент, аргументи, результат і тривалість кожного виклику.

3. **Написати тести схем на двох рівнях** (30 хв, ауд.):

- відсутнє обов'язкове поле і поле неправильного типу не проходять схему — це протокольна помилка JSON-RPC, а не виконання з дефолтами;
- семантично хибні дані (неіснуюча дата, невідомий ідентифікатор) схему проходять і повертають результат із `isError: true` та повідомленням, з якого випливає наступна дія;
- окремим тестом перевірити виклик невідомого інструмента.

4. **Скласти набір задач** (50 хв, сам.): 40 задач над цими даними з відомими правильними відповідями. Набір **фіксується до експериментів**.

5. **Реалізувати дві конфігурації** (60 хв, ауд.) з однаковими інструментами і однаковою моделлю: одиночний агент і supervisor з двома виконавцями (наприклад, пошук і перевірка цитат).

Тут є спокуса зробити supervisor «розумнішим», давши йому кращий промпт. Тоді порівняння втрачає сенс: різнитися має рівно одна річ — топологія.

6. **Прогнати експеримент** (40 хв, сам.): обидві конфігурації по три рази на всіх 40 задачах. Виміряти частку розв'язаних, `pass`³, точність вибору інструментів, токени й затримку на задачу, кількість помилок координації.

Перевірка: рахуйте не лише частку успіху, а й токени — саме вони зазвичай і пояснюють різницю.

7. **Порахувати довірчі інтервали** (30 хв, ауд.). Інтервали Вілсона — описово для кожної конфігурації; висновок робиться за 95 % інтервалом *парної різниці* (bootstrap, ресемплінг за задачами, три прогони всередині задачі усереднюються). Письмово відповісти, чи накриває інтервал різниці нуль і чому перекриття маргінальних інтервалів висновком не є.

8. **Реалізувати й відкалібрувати суддю** (50 хв, сам.) на основі моделі: розмітити вручну 20 випадків, порахувати узгодженість із людською міткою. Якщо узгодженість низька, змінити інструкцію судді й повторити.

Перевірка: звітується капша, а не сира частка збігів.

9. **Зберегти траси** (40 хв, сам.) у форматі JSONL з атрибутами за конвенціями OpenTelemetry: система, операція, вхідні й вихідні токени, ідентифікатор сесії. Показати, як за трасою відновлюється шлях однієї невдалої задачі.
10. **Здати роботу**: код сервера і клієнта, набір задач, таблицю порівняння двох конфігурацій із довірчими інтервалами, результати калібрування судді, приклад траси і висновок до 200 слів із явним рішенням, чи виправдана мультиагентна конфігурація.

Відповідь «мультиагентна конфігурація не виправдана» є повноцінним результатом роботи, якщо вона підкріплена інтервалом різниці й таблицею витрат.

Контрольні запитання

1. Як протокол перетворює $M \times N$ інтеграцій на $M + N$?
2. Чим ресурс відрізняється від інструмента в термінах ініціативи?
3. Чому підключення MCP-сервера не є перевіркою його безпеки?
4. У чому різниця між $\text{pass}@k$ і pass^k і чому для надійності важлива друга?
5. Конфігурація А розв'язала 31 задачу з 40, конфігурація В — 33. Що з цього випливає?
6. Які три зсуви властиві судді на основі моделі?
7. Навіщо калібрувати суддю, якщо він і так узгоджується з інтуїцією?
8. Які метрики фіксують координаційну вартість мультиагентної системи?
9. Чому набір задач і конфігурації мають бути однаковими при порівнянні?
10. Що саме треба записати у трасу, щоб через тиждень відтворити невдалий прогін?

Література

1. Zheng L. та ін. [Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena](#). NeurIPS, 2023.
2. Cemri M. та ін. [Why Do Multi-Agent LLM Systems Fail?](#) arXiv:2503.13657, 2025.

3. DeBenedetti E. та ін. [AgentDojo](#). NeurIPS, 2024.
4. Yao S. та ін. [\$\tau\$ -bench: A Benchmark for Tool-Agent-User Interaction](#). arXiv:2406.12045, 2024.